

Lab02-CopyDir

目标

通过一个实际的复制程序来加深对操作系统的理解，明白操作系统如何为用户程序提供支持（各种系统调用）

1. 写一个c语言程序，实现一个目录及其子目录的复制
2. 测试目录，最新的稳定版linux内核linux-6.17.1.tar.xz
 - a. 下载：wget <https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.17.1.tar.xz>
 - b. 解压缩到linux-6.17.1：tar -Jxvf linux-6.17.1.tar.xz
3. 实现C语言程序，复制linux-6.17.1到linux-6.17.1-[学号]
4. 验证复制后目录的正确性

工具

编译器：GCC

代码块

```
1 sudo apt-get install build-essential
```

代码编辑器：vim

代码块

```
1 vim mycopy.c
```

IDE：也可选择其它可视化的代码编辑器

文件修改检测工具：md5sum

代码块

```
1 md5sum fileA fileB
```

获得程序的执行时间：time命令

```
1 time pwd
```

编写程序可能用到的库函数

1. 目录操作：opendir、readdir、closedir、mkdir。
2. 文件操作：fopen、fclose、fread、fwrite。
3. 路径拼接与字符串处理：snprintf、strcmp。
4. 文件信息判断：stat、S_ISDIR、S_ISREG。
5. 错误处理：perror。

下面简单介绍：

1. fopen

- **作用：**打开文件，得到一个 `FILE*` 指针用于读写文件内容
- **原型：** `FILE *fopen(const char *filename, const char *mode);`
- **主要参数：**
 - `filename`：文件名（路径）
 - `mode`：打开模式，例如 `"rb"`（读二进制），`"wb"`（写二进制）

2. fclose

- **作用：**关闭打开的文件
- **原型：** `int fclose(FILE *stream);`
- **主要参数：**
 - `stream`：由 `fopen` 获得的 `FILE*` 指针

3. fread

- **作用：**从文件读取数据到内存
- **原型：** `size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);`
- **主要参数：**
 - `ptr`：缓冲区指针
 - `size`：每个元素的字节数
 - `nmemb`：元素数量
 - `stream`：文件指针

4. fwrite

- **作用：**将内存的数据写入文件

- **原型：**`size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);`

- **主要参数：**

- `ptr`：数据缓冲区指针
- `size`：每个元素的字节数
- `nmemb`：元素数量
- `stream`：文件指针

5. `opendir`

- **作用：**打开一个目录，返回 `DIR*` 用于遍历目录内容

- **原型：**`DIR *opendir(const char *name);`

- **主要参数：**

- `name`：目录路径

6. `readdir`

- **作用：**读取一个目录项，返回 `struct dirent*`

- **原型：**`struct dirent *readdir(DIR *dirp);`

- **主要参数：**

- `dirp`：目录指针（由 `opendir` 获得）

7. `closedir`

- **作用：**关闭一个打开的目录

- **原型：**`int closedir(DIR *dirp);`

- **主要参数：**

- `dirp`：目录指针（由 `opendir` 获得）

8. `stat`

- **作用：**获取文件/目录的属性（如类型、权限、大小等）

- **原型：**`int stat(const char *path, struct stat *buf);`

- **主要参数：**

- `path`：文件或目录路径
- `buf`：结构体指针，文件信息存放在 `struct stat` 中

9. `mkdir`

- **作用：**新建一个目录

- **原型:** `int mkdir(const char *pathname, mode_t mode);`

- **主要参数:**

- `pathname` : 新目录路径
- `mode` : 权限 (如 `0755` 表示 `rwxr-xr-x`)

10. `snprintf`

- **作用:** 格式化字符串到缓冲区 (安全版本, 防止溢出)

- **原型:** `int snprintf(char *str, size_t size, const char *format, ...);`

- **主要参数:**

- `str` : 目标缓冲区
- `size` : 缓冲区大小
- `format` : 格式串 + 后面参数

11. `strcmp`

- **作用:** 比较字符串

- **原型:** `int strcmp(const char *s1, const char *s2);`

- **主要参数:**

- `s1`、`s2` : 两个字符串指针

12. `perror`

- **作用:** 打印最近一次标准库函数错误信息

- **原型:** `void perror(const char *s);`

- **主要参数:**

- `s` : 自定义前缀 (可为空字符串)

13. `printf`

- **作用:** 输出格式化文本到屏幕

- **原型:** `int printf(const char *format, ...);`

14. `popen` / `pclose`

- **作用:** 执行外部命令, 并读取输出

- **原型:**

- `FILE *popen(const char *command, const char *type);`
- `int pclose(FILE *stream);`

- **主要参数:**

- `command`：要执行的命令（如 `"md5sum filename"`）
- `type`：`"r"` 读取输出，`"w"` 写入命令

15. `fgets`

- **作用**：从文件或标准输入读取一行
- **原型**：`char *fgets(char *s, int size, FILE *stream);`
- **主要参数**:
 - `s`：目标缓冲区
 - `size`：缓冲区大小
 - `stream`：文件指针

16. `sizeof`

- **作用**：获取变量/类型所占字节数

可能用到的头文件

上述函数分别在这些头文件中声明：

- `<stdio.h>`：文件与输入输出（`fopen`, `fclose`, `fread`, `fwrite`, `popen`, `pclose`, `perror`, `printf`, `fgets`）
- `<string.h>`：字符串处理函数（`strcmp`, `snprintf`）
- `<sys/stat.h>`、`<sys/types.h>`：文件属性结构体及类型（`stat`, `mkdir`）
- `<dirent.h>`：目录相关（`opendir`, `readdir`, `closedir`）
- `<unistd.h>`：Unix/Linux标准函数
- `<errno.h>`：错误码

代码块

```
1  struct dirent {
2      ino_t d_ino;           // d_ino 此目录项的 inode 号（唯一标识文件的数字）
3      off_t d_off;          // d_off 目录文件开头至此目录项的偏移量
4      unsigned short d_reclen; // d_reclen 该目录项的长度（包括 d_name），用于遍历目录
                               时定位下一个目录项
5      unsigned char d_type;  // d_type 文件类型（如普通文件、目录、符号链接等）
6      char d_name[256];     // d_name 文件名称，最长256字节（不包含结尾的\0）
7  };
```

实验流程

1. 写一个c语言程序复制一个目录及其子目录，并且验证复制结果
 - a. 拷贝一个指定目录及其子目录到另外一个目录中
 - i. 不用处理符号链接、特殊文件（如设备文件），仅复制普通文件和子目录即可
 - b. 检验源目录和目标目录的内容是否一致
 - c. 并采用Python等语言编写拷贝目录的程序，与C语言拷贝目录程序进行性能对比，并分析原因
2. 一个遍历目录的例子
 - a. 撰写程序listdir.c

代码块

```
1  #include <stdio.h>
2  #include <dirent.h>
3  #include <unistd.h>
4  #include <stdlib.h>
5
6  int main()
7  {
8      DIR * dir;
9      struct dirent * ptr;
10     /*open dir*/
11     dir = opendir("/home");
12     /*read dir entry*/
13     while((ptr = readdir(dir)) != NULL)
14     {
15         printf("d_name : %s", ptr->d_name);
16         if (ptr->d_type==DT_DIR){
17             printf("\tDir");
18         }
19         printf("\n");
20     }
21     /*close dir*/
22     closedir(dir);
23     exit(0);
24 }
```

b. 编译程序

代码块

```
1  gcc listdir.c -o listdir
```

c. 运行

代码块

```
1 ./listdir
```

3. 撰写实验报告，包括程序代码，程序代码流程说明，代码执行结果及时间，不同语言复制性能对比分析，实验中遇到的其它问题等内容